╰(..:: JSON (JavaScript Object Notation ::..)╯

~ By Sagar Biswas

💣 Part 1: Introduction

JSON (JavaScript Object Notation) is a lightweight data-interchange format that is easy for humans to read and write and easy for machines to parse and generate. It is widely used for data exchange between a client (ex: a browser or mobile app) and a server. Lets dive into the details of JSON.

1. Why JSON?

Imagine a server that stores weather data. Developers working with different programming languages (ex: C++, Java, JavaScript) need access to this data. If the server provides the data in **JSON format**, all these developers can easily use it because JSON is:

- **Text-based** : Stored as plain text and can be read by any programming language.
- **Human-readable** : Easy to understand and debug.
- **Lightweight** : Efficient for data transmission.
- **Language-independent** : Supported by all modern programming languages.
- **File extension** : .json.
- **Media type** : application/json.

JSON is commonly used for:

- **Data exchange** between a client and a server.
- **Configuration files** (ex: .json files).
- **Storing structured data** (ex: user profiles, product catalogs).

2. What is JSON?

JSON stands for **JavaScript Object Notation**. It follows JavaScript's syntax for object representation but is language-independent.

3. JSON Basic Structure

JSON data can be stored in **two** structures:

1. **Objects** : Enclosed in {} and consist of key-value pairs.
2. **Arrays** : Enclosed in [] and consist of a list of values.

..:: Note : JSON does not support comments.

☠ **Example of a JSON Object:**

```
{
    "name": "Sagar Biswas",
    "age": 22,
    "languages": ["Hindi", "English", "Bangla"]
}
```

☠ **Example of a JSON Array:**

```
[
    "Hindi",
    "English",
    "Bangla"
]
```

💧 **Nested JSON:**

JSON objects and arrays can be nested within each other.

☠️ **Example:**

```
{
    "name": "Sagar Biswas",
    "age": 22,
    "address": {
        "city": "Dhaka",
        "country": "Bangladesh"
    },
    "languages": ["Hindi", "English", "Bangla"]
}
```

4. JSON Syntax (Key-Value Format)

Rules for JSON Syntax:

1. **Objects**:
    - Start with { and end with }.
    - Keys must be in **double quotes** ("key").
    - Values can be **strings**, **numbers**, **booleans**, **objects**, **arrays**, or **null**.
2. **Arrays**:
    - Start with [and end with].

..:: Note : **Objects** & **Arrays** Elements are separated by commas (,).

Example:

```
{
    "name": "Sagar Biswas",
    "age": 22,
    "isStudent": true,
    "languages": ["Hindi", "English", "Bangla"],
    "address": {
        "city": "Dhaka",
        "country": "Bangladesh"
    }
}
```

5. JSON Data Types

JSON supports the following data types:

1. **String** : "Hello World"
2. **Number** : 42 (integer) or 3.14 (float)
3. **Boolean** : true or false
4. **Null**: null
5. **Object** : { "key": value }
6. **Array** : [value1, value2, ...]

Example Using All Data Types:

```
{
    "active": true,
    "deactivated": false,
    "lastName": null,
    "user": {
        "name": "Sagar Biswas",
```

```
        "age": 22,
        "cgpa": 3.28
    },
    "languages": [
        "Hindi",
        "English",
        "Bangla"
    ]
}
```

6. JSON vs JavaScript Objects

Feature	JavaScript Object	JSON Object
Keys	Keys don't need quotes.	Keys must be in double quotes ("").
Strings	Supports single (') and double (") quotes.	Only double quotes ("") are allowed.
Data Types	Supports functions, undefined, etc.	Limited to strings, numbers, booleans, objects, arrays, and null.

☠ **JavaScript Object Example:**

```
let student = {
    name: "Sagar Biswas",
    age: 22,
    cgpa: 3.28
};
```

☠ **Equivalent JSON Format:**

```
{
    "name": "Sagar Biswas",
    "age": 22,
    "cgpa": 3.28
}
```

Part 2: Demo: Creating a JSON File

☠ **Example: Student Data**

Here is an example of a JSON file that stores student data, including nested objects and arrays.

JSON File:

```
{
    "students": [
        {
            "name": "Sagar Biswas",
            "middleName": null,
            "age": 22,
            "cgpa": 3.28,
            "isRegistered": true,
            "languages": [
                "Hindi",
                "English",
                "Bangla"
            ],
            "Friends": [
                {
                    "name": "Mr. NoBody",
                    "age": 22
                },
                {
                    "name": "Shisher Biswas",
                    "age": 24
```

```
                }
            ]
        },
        {
            "name": "Shisher Biswas",
            "middleName": null,
            "age": 24,
            "cgpa": 3.88,
            "isRegistered": false,
            "languages": [
                "Hindi",
                "English",
                "Bangla",
                "Spanish"
            ],
            "Friends": [
                {
                    "name": "Sagar Biswas",
                    "age": 22
                },
                {
                    "name": "Shamim",
                    "age": 24
                }
            ]
        }
    ]
}
```

Part 3: JSON Data Validation

💧 Why Validate JSON?

JSON data validation ensures that the JSON structure and content meet specific requirements. This is crucial for:

- **Data integrity** : Ensuring the data is correct and consistent.
- **Error prevention**: Catching errors early in the data pipeline.
- **Compatibility** : Ensuring the JSON conforms to a predefined schema.

💧 Tools for JSON Validation

1. **JSON Beautifier**:
   - A tool to format and validate JSON data.
   - Visit: https://codebeautify.org/jsonviewer.
2. **JSON Schema Validator**:
   - Validates JSON against a schema.
   - Visit: https://www.jsonschemavalidator.net/.

Part 4: JSONPath

What is JSONPath?

JSONPath is a query language for JSON, similar to XPath for XML. It allows you to navigate and manipulate JSON data.

Key Features:

- **Access** : Retrieve specific data from JSON.

- **Modify** : Update JSON data.
- **Delete** : Remove data from JSON.
- **Convert** : Transform JSON data into other formats.

💧 Accessing JSON Data

1. Using Dot Notation

Dot notation is used to access properties of JSON objects.

☠ **Example:**

```
var data = {
    "studentsData": [
        { "name": "Sagar Biswas", "age": 22, "city": "Dhaka" },
        { "name": "Shisher Biswas", "age": 24, "city": "Motijheel" }
    ],
    "courses": ["HTML", "CSS", "JavaScript", "Python"]
};

console.log(data.studentsData[0].name); // Output: Sagar Biswas
console.log(data.courses[2]); // Output: JavaScript
```

2. Using Bracket Notation

Bracket notation is used to access properties dynamically or when the key contains special characters.

☠ **Example:**

```
console.log(data["studentsData"][1]["name"]); // Output: Shisher Biswas
console.log(data["courses"][3]);  // Output: Python
```

..:: Tips: <u>use dot for objects and bracket for arrays</u>

3. Looping Through JSON Data

☠ **Example:**

```
for (let key in data) {
    console.log();
    console.log("Key: ", key);
    console.log("Value: ", data[key]);
}
```

Output:

```
Index: students
Data: [
  { name: 'Sagar Biswas' },
  { name: 'Abhishek Biswas', age: 24 }
]
```

4. Printing whole object

```
console.log("\nData: ", data); // printing the whole object
```

💧 Modifying JSON Data

Example:

```
var studentsData = {
    "students": [
        {
            "name": "Sagar Biswas",
            "age": 22
        },
```

```
        {
            "name": "Shisher Biswas",
            "age": 24
        }
    ]
};

// Modify the name of the second student
studentsData.students[1].name = "Abhishek Biswas";
console.log(studentsData.students[1].name); // Output: "Abhishek Biswas"
```

💧 Deleting JSON Data

Example:

```
// Delete the age property of the first student
delete studentsData.students[0].age;
console.log(studentsData.students[0].age); // Output: undefined
```

☠️ Practical Example: Working with JSON Data

File Name: studentsData.json

```
{
    "students": [
        {
            "name": "Sagar Biswas",
            "middleName": null,
            "age": 22,
            "cgpa": 3.28,
            "isRegistered": true,
            "languages": [
                "Hindi",
                "English",
                "Bangla"
            ],
            "Friends": [
                {
                    "name": "Mr. NoBody",
                    "age": 22
                },
                {
                    "name": "Shisher Biswas",
                    "age": 24
                }
            ]
        },
        {
            "name": "Shisher Biswas",
            "middleName": null,
            "age": 24,
            "cgpa": 3.88,
            "isRegistered": false,
            "languages": [
                "Hindi",
                "English",
                "Bangla",
                "Spanish"
            ],
            "Friends": [
                {
                    "name": "Sagar Biswas",
                    "age": 22
                },
                {
                    "name": "Shamim",
                    "age": 24
                }
            ]
```

```
            }
        ],
        "data": {
            "name": "Sagar Biswas",
            "age": 22,
            "city": "Dhaka"
        }
    }
```

File Name: script.js

```
var studentsData = require("./studentsData.json");

console.log("\n");
var length = studentsData.students.length; // useful for loops
console.log("--> Total Number of students:", length);
console.log("\n");

// Accessing data
console.log("..:: ACCESSING DATA ::..");
console.log("\n\n1st student name:", studentsData.students[0].name);
console.log("2nd student name:", studentsData.students[1].name);

console.log("\n\n1st student second languages:", studentsData.students[0].languages[1]);
console.log("2nd student 4th languages:", studentsData.students[1].languages[3]);

console.log("\n\n1st student second friend's age:", studentsData.students[0].Friends[1].age);
console.log("2nd student friends second friend's name:", studentsData.students[1].Friends[0].name);

console.log("\n");

// Modifying data
console.log("..:: MODIFYING DATA ::..\n");
console.log("\n2nd student name:", studentsData.students[1].name);
studentsData.students[1].name = "Abhishek Biswas";
console.log("After modifying 2nd student name:", studentsData.students[1].name);

console.log("\n\n1st student second friend's age:", studentsData.students[0].Friends[1].age);
studentsData.students[0].Friends[1].age = 25;
console.log("After modifying 1st student second friend's age:",
studentsData.students[0].Friends[1].age);

console.log("\n");

// Deleting data
console.log("..:: DELETING DATA ::..\n");
console.log("\n2nd student name:", studentsData.students[1].name);
delete studentsData.students[1].name;
console.log("After deleting 2nd student name:", studentsData.students[1].name);

console.log("\n\n1st student second friend's age:", studentsData.students[0].Friends[1].age);
delete studentsData.students[0].Friends[1].age;
console.log("After deleting 1st student second friend's age:",
studentsData.students[0].Friends[1].age);

// Looping data
for (x in studentsData) {
    console.log("\n\nIndex: ", x);
    console.log("Index's Data: ", studentsData[x]);
}

console.log("\n\n");
```

💧 Tools for JSONPath

1. **JSONPath Finder**:
    - A tool to visualize and query JSON data.
    - Visit: https://jsonpathfinder.com/.

2. **JSON Beautifier and Editor (Chromium Extension)**:
    - A browser extension for formatting and editing JSON.
    - Visit: https://chromewebstore.google.com/detail/json-beautifier-and-edito/lpopeocbeepakdnipejhlpcmifheolpl?hl=en.

Part 5: Conversion: JavaScript Objects to JSON and Vice Versa

When working with data in web applications, it's common to transfer data between the client (browser) and the server. JavaScript provides two key methods for this purpose:

1. **JSON.stringify():** Converts a JavaScript object into a JSON string. This is useful when sending data to a server.
2. **JSON.parse():** Converts a JSON string back into a JavaScript object. This is useful when receiving data from a server.

☠️ **Example: Conversion Between JavaScript Objects and JSON**

```
const obj = {
    "data": {
        "name": "Sagar Biswas",
        "age": 22,
        "city": "Dhaka"
    }
};

// Convert JavaScript object to JSON string
const jsonString = JSON.stringify(obj);
console.log(typeof jsonString); // Output: "string"
console.log(jsonString); // Output: {"data":{"name":"Sagar Biswas","age":22,"city":"Dhaka"}}

// Convert JSON string back to JavaScript object
const parsedObj = JSON.parse(jsonString);
console.log(typeof parsedObj); // Output: "object"
console.log(parsedObj); // Output: { data: { name: 'Sagar Biswas', age: 22, city: 'Dhaka' } }
```

Part 6: Schema: Defining the Structure of JSON Data

A **schema** is a blueprint or standard format that defines the structure, data types, and constraints of JSON data. It ensures that the data adheres to a specific format, making it easier to validate and process.

💧 **What is a Schema?**

- A schema describes the expected structure of JSON data, including:
    - **Data types** (ex: string, number, boolean).
    - **Required fields**.
    - **Nested objects** and **arrays**.
- It is commonly used for **validating client-submitted data** to ensure it meets the required format before processing.

☠️ **Example: JSON Schema**

```
{
    "data": {
        "name": "Sagar Biswas",
        "age": 22,
        "city": "Dhaka"
    },
    "schema": {
        "$schema": "http://json-schema.org/draft-04/schema#",
        "type": "object",
```

```
        "properties": {
            "data": {
                "type": "object",
                "properties": {
                    "name": {
                        "type": "string"
                    },
                    "age": {
                        "type": "integer"
                    },
                    "city": {
                        "type": "string"
                    }
                },
                "required": ["name", "age", "city"]
            }
        },
        "required": ["data"]
    }
}
```

Explanation:

- $schema : Specifies the version of the JSON Schema standard being used.
- Type : Defines the type of the root object (in this case, an object).
- Properties : Describes the structure of the data object, including the expected data types for each field.
- Required : Lists the fields that must be present in the data object.

💧 **Why Use a Schema?**

1. **Validation** : Ensures that the data submitted by the client adheres to the expected format.
2. **Consistency** : Maintains a consistent structure for JSON data across the application.
3. **Error Prevention** **ss**: Catches errors early by validating data before processing.

💧 **Tools for Working with JSON Schemas**

1. **JSON to Schema Converter**:
   - Converts a JSON object into a schema automatically.
   - Website: https://www.liquid-technologies.com/online-json-to-schema-converter
2. **JSON Schema Validator**:
   - Validates JSON data against a schema to ensure it meets the required format.
   - Website: https://www.jsonschemavalidator.net/

Part 7: Practical Use Case: Validating Client-Submitted Data

Imagine a scenario where a client submits data to a server. The server needs to validate the data before processing it. Here's how a schema can help:

Step 1: Define the Schema

```
{
    "$schema": "http://json-schema.org/draft-04/schema#",
    "type": "object",
    "properties": {
        "name": { "type": "string" },
        "age": { "type": "integer" },
        "city": { "type": "string" }
    },
    "required": ["name", "age", "city"]
```

```
}
```

Step 2: Validate the Data

```
const schema = {
    "$schema": "http://json-schema.org/draft-04/schema#",
    "type": "object",
    "properties": {
        "name": { "type": "string" },
        "age": { "type": "integer" },
        "city": { "type": "string" }
    },
    "required": ["name", "age", "city"]
};

const data = {
    "name": "Sagar Biswas",
    "age": 22,
    "city": "Dhaka"
};

// Validate the data against the schema
const isValid = validateData(data, schema); // Assume `validateData` is a validation
function
console.log(isValid); // Output: true
```

Explanation:

- The schema ensures that the data object contains the required fields (name, age, city) and that each field has the correct data type.
- If the data doesn't match the schema, the validation will fail, and the server can reject the request.

Summary

- JSON.stringify() and JSON.parse() are essential for converting between JavaScript objects and JSON strings.
- A **schema** defines the structure and constraints of JSON data, ensuring consistency and validity.
- Tools like **JSON to Schema Converter** and **JSON Schema Validator** simplify the process of creating and validating schemas.

---------- x ----------